

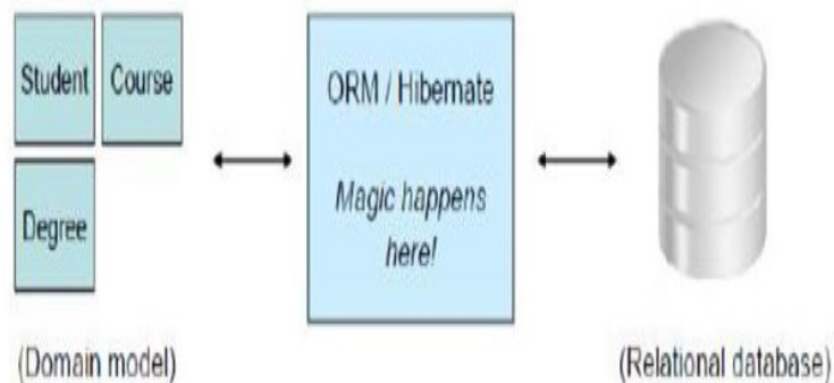
ORM – Mapping a Java Object to Database table without JDBC hard coding.

Object – Relational Mapping

Provides a simple API for storing and retrieving Java objects directly to and from the database

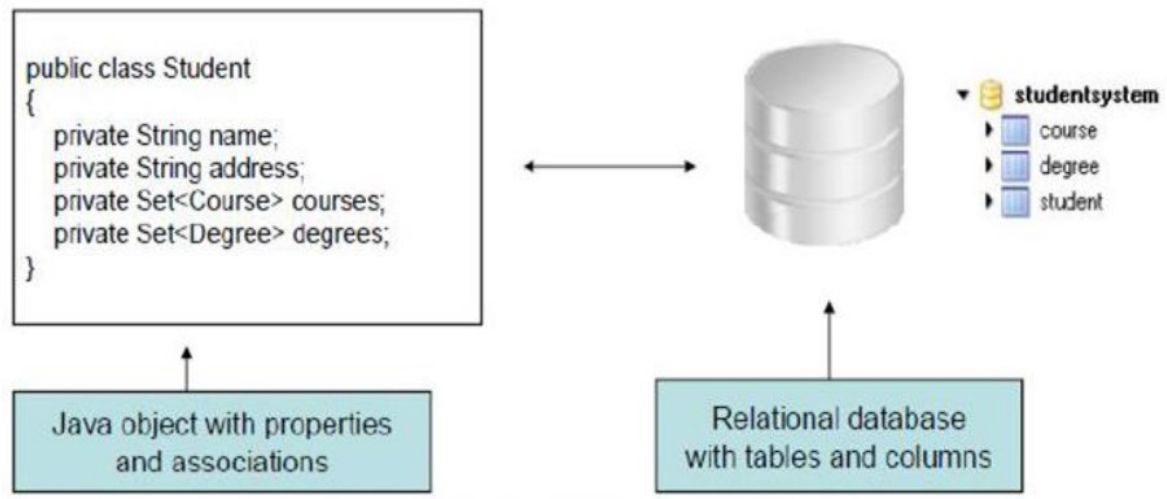
No need to follow specific rules or design patterns

Transparent: Direct data manipulation in RDBMS using an OOP Language



- Object model and relational model mismatch

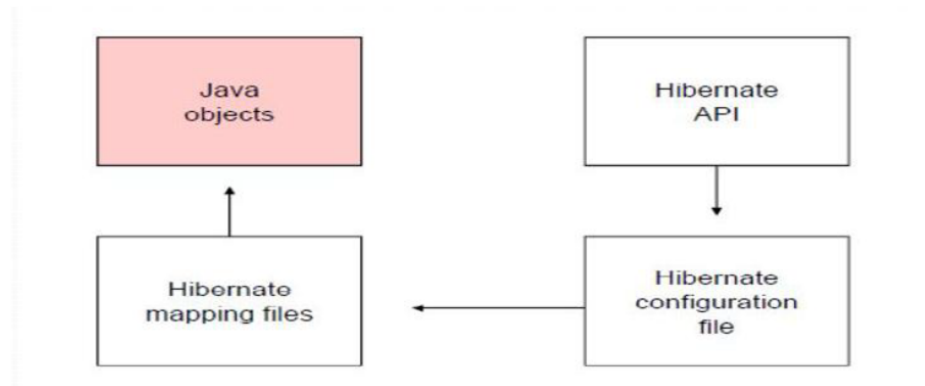
- Mapping is an issue



Hibernate – Why?

- Database independent application
- Avoid writing queries
- Avoid JDBC API completely
- Hibernate uses connection pooling technique
- Automatic Key Generation
- Develop the Application in short Period of time

Hibernate Components



Hibernate configuration files is to set the properties to connect to database server.

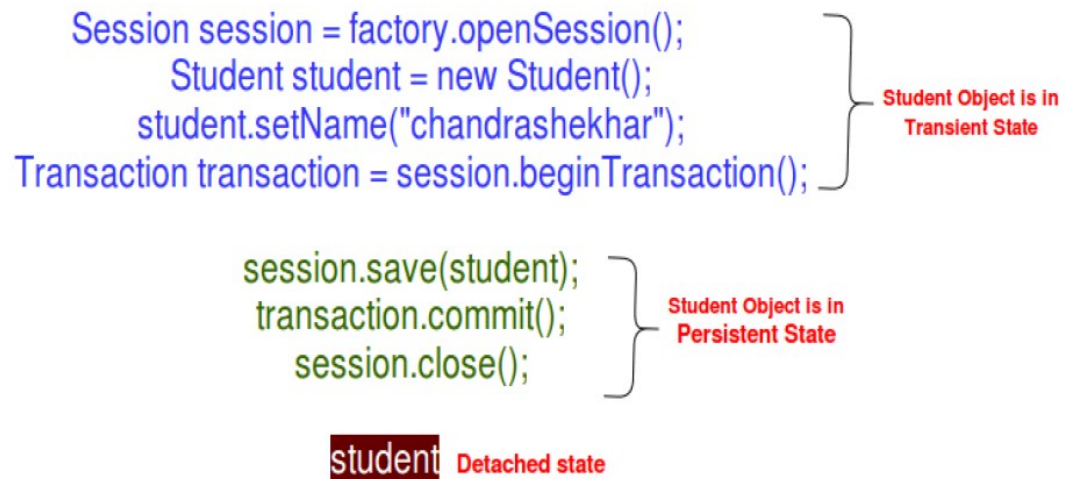
```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver
    </property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost/hibernate
    </property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password"></property>
```

Hibernate Mapping files is to map a java class to db table, class properties to db table columns

```
<hibernate-mapping>
  <class name="Example.Contact" table="CONTACT">
    <id name="id" type="long" column="ID" >
      <generator class="assigned"/>
    </id>

    <property name="firstName">
      <column name="FIRSTNAME" />
    </property>
```

States of Java Objects in ORM



Maven Project :

Apache Maven is an open-source **build automation and project management tool** primarily used for Java applications. It simplifies project builds, dependency management using a standard structure

Maven uses the pom.xml (Project Object Model) file to define project information, dependencies(jar files substitution)

Maven automatically manages external .jar files (dependencies) by downloading them from repositories such as Maven Central based on the entries in the pom.xml file, eliminating the need to manually add JARs to the project.

****Demo maven-orm-app**

Hibernate Query Language

- HQL is similar to SQL
- Does not depend on database table
- Uses class names instead of table names
- Hence, DB independent query language



** Demo maven-orm-hql

JPA – Java Persistence API :

- **JPA** is a Java specification to map Java objects to relational database tables using Object-Relational Mapping (ORM).
- **Database Independence:** JPA allows developers to write database-independent persistence code, making it easier to switch between databases with minimal changes.
- **Annotations and XML Mapping:** JPA supports mapping entities using annotations (e.g., @Entity, @Id, @Column) as well as XML configuration files.
- **CRUD Operations:** JPA simplifies database operations such as **Create, Read, Update, and Delete (CRUD)** through the EntityManager API, reducing the need for manual SQL.
- **Implemented by ORM Frameworks:** JPA is only a **specification**, not an implementation. Popular ORM frameworks such as Hibernate, EclipseLink, and Apache OpenJPA provide JPA implementations.

CRUD Methods

Operation	JPA Method
Create	persist()
Read	find()
Update	Find the existing data after retrieval. Modify and commit
Delete	remove()

JPQL (Java Persistence Query Language) is the standard query language defined by JPA for retrieving, updating, and deleting data. It works with entity classes and their fields, rather than directly using database tables and columns.

JPQL is database-independent and portable

We need to create persistence.xml file in META-INF folder.

META-INF folder should be created in "src/main/resources" folder

Criteria API is a type-safe, object-oriented query API provided by JPA for creating dynamic database queries programmatically.

Criteria queries are best suited for dynamic search conditions where filters can change at runtime

Eg. // mysql>SELECT * FROM EMPLOYEE WHERE salary > 50000

cq.select(employee).where(cb.gt(employee.get("salary"), 50000));

JPA Entity States describe the lifecycle of an entity object as it moves through the application and database. The four states are New (Transient), Managed (Persistent), Detached, and Removed.

Eager loading :

Eager Loading is a fetching strategy in Hibernate where the related (associated) objects are loaded **immediately along with the parent object**

Advantage: Data is readily available without additional database queries.

Lazy loading :

Lazy Loading is a fetching strategy in Hibernate where the related (associated) objects are **not loaded immediately**. They are fetched from the database **only when they are accessed for the first time**.

Advantage: Improves performance and reduces memory usage by loading only the required data.

One-to-One Mapping (@OneToOne)

Maps one entity to exactly one related entity. For example, one **Employee** has one **Passport**, and one passport belongs to one employee.

One-to-One Relationship / Mapping

Scenario

Each **Customer** has exactly **one Address**.

Customer 1 ----- 1 Address

One application that creates both Customer and Address tables.

CUSTOMER

CUSTOMER_ID

MOBILE

ADDRESS_ID (FK)

ADDRESS

ADDRESS_ID - PK

CITY

STATE

COUNTRY

CUSTOMER

CUSTOMER_ID

CUSTOMER_NAME

MOBILE

ADDRESS_ID (FK)

Spring framework

Spring Framework is a lightweight, open-source Java framework used to develop enterprise applications. **An application framework.**

Spring provides Inversion of Control (IoC) and Dependency Injection (DI), allowing the framework to manage object creation and dependencies, resulting in loosely coupled

Spring offers multiple modules such as **Spring Core**, **Spring MVC**, **Spring Data**, **Spring Security** enabling developers to build web, RESTful, database-driven apps.

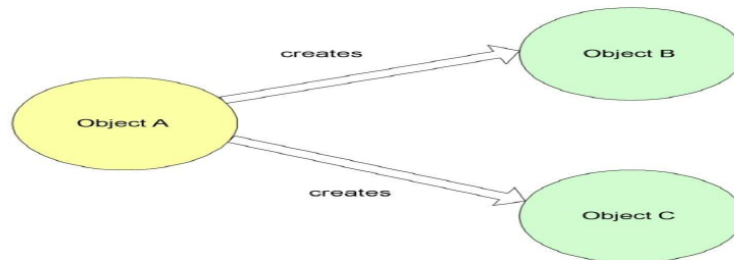
Spring follows the POJO (Plain Old Java Object) programming model and supports annotation-based as well as XML-based configuration

Dependency Injection & Inversion Of Control

Dependency : A POJO / bean / Entity

Non IOC – Getting a POJO by pulling – Hard Coding

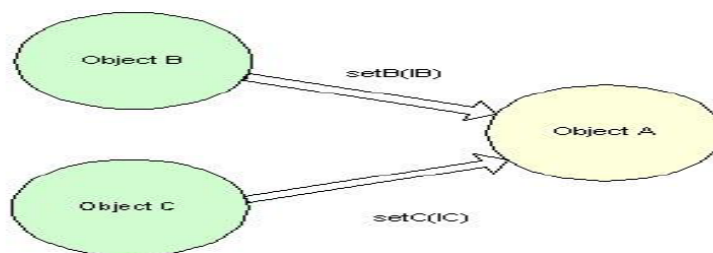
```
Employee emp1 = new Employee();  
Employee emp2 = new Employee();  
// Explicit creation
```



Inversion of Control

IOC : Getting a POJO Object by Pushing on demand by
Spring Core Container

```
Employee emp1 = (Employee)xxxxx.getBean("emp1");  
Employee emp2 = (Employee)xxxxx.getBean("emp2");
```



The act of Dependency Injection is known as 'wiring'.

Spring Core Container

Spring's Container uses IoC to manage components of the application.

- **Application context**
(org.springframework.context.ApplicationContext)
provides application framework services

Spring way of DI

- Java classes should be as independent as possible from each other
- Spring Framework injects these dependencies via the container.
- Piecing together all beans in the Spring Container is called **wiring**.
- Wiring can be done through xml.



```
<?xml version= "1.0" encoding= "UTF-8"?>
<beans xmlns= "http://www.springframework.org/schema/beans"
  xmlns:xsi= "http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation= "http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

  <bean id= "helloWorld" class= "com.mycompany.springcore.hw.HelloWorld">
    <property name= "wish" value= "Hello World !"/>
  </bean>

</beans>
```

```
// not pulling the bean - // injecting the bean
// HelloWorld hw = new HelloWorld() -- pulling the bean
HelloWorld helloWorld =
    (HelloWorld) context.getBean("helloWorld");
// Invoke the method
//Setter injection is applied. Default injection is setter injection
System.out.println(helloWorld.getWish());
```

****Demo - maven-springcore-helloworld**

Injecting Beans in Spring Framework

Constructor Injection

Scenario : **Employee HAS-A Address.**

We can inject the dependency by constructor.

When using constructor injection, **setters are not used !**

The **<constructor-arg>** sub element of **<bean>** tag is used for constructor injection.

```
<!-- Account beans -->
<bean id="account1" class=
"com.mycom.springcore.constructorinjection.Account">
    <constructor-arg value="ACC12345"/>
    <constructor-arg value="500000"/>
</bean>

<!-- Customer beans with constructor injection -->
<bean id="customer1" class=
"com.mycom.springcore.constructorinjection.Customer">
    <constructor-arg ref="account1"/>
</bean>
```

**** Demo - maven-springcore-constructor-injection**

Spring Annotations :

@Bean: Placed on methods inside a `@Configuration` class to indicate that the return value should be registered as a Spring bean.

@Autowired: Tells Spring to inject a dependency automatically by matching type auto wiring. Can be applied to constructors (recommended), fields, or setters.

@Configuration: Indicates that a class declares one or more `@Bean` methods. Spring processes these classes to generate bean definitions at runtime.

// To scan all components without applicationContext.xml file

@ComponentScan("com.bank") – scans all beans from com.bank.xxxx packages

Setter Injection

Scenario : **Employee HAS-A Address.**

The Address class object will be termed as the **Dependent** object.

The Employee class object will be termed as the **Dependency**

To avoid xml configuration, Java class configuration is used in Spring.

Eg.

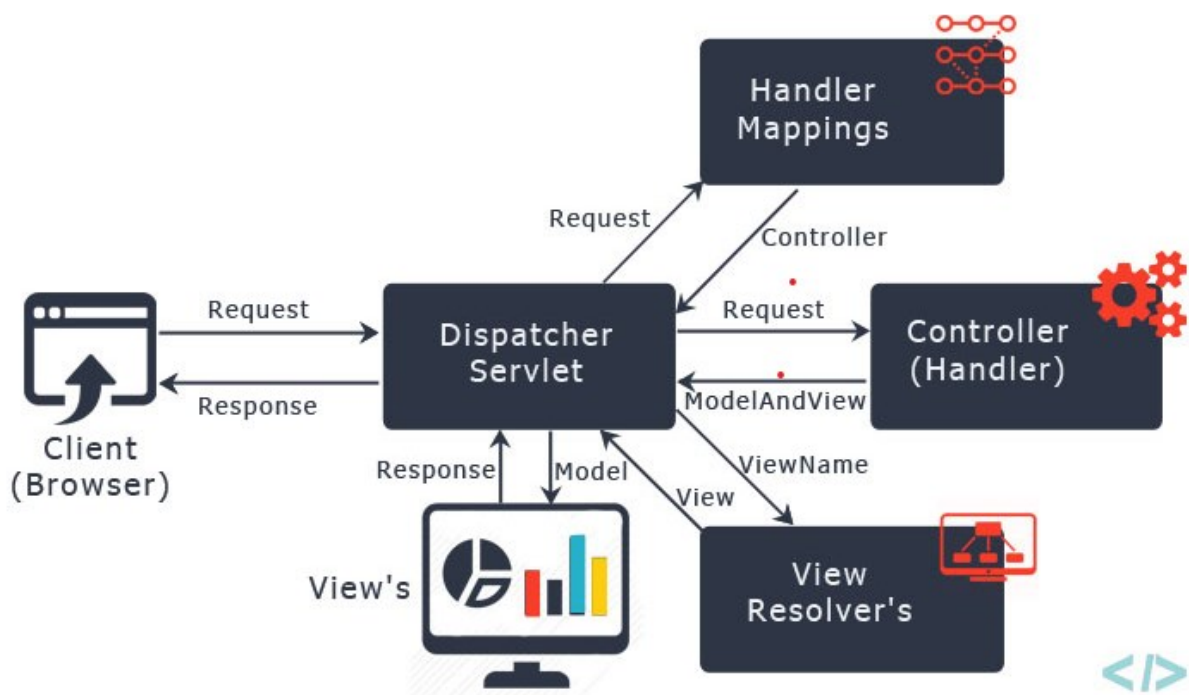
```
@Bean
@Scope(value = ConfigurableBeanFactory.SCOPE_SINGLETON)
public Address address(){
    return new Address(101,"Signature Casle","Krishna Nagar", "Pune");
}
@Bean
public Employee employee(){
    return new Employee(1111,"Anoohya");
}
```

**** Demo - maven-springcore-setter-injection.**

Spring MVC Framework

Spring MVC

- Spring provides a very clean division between
 - Controllers, Java Bean Models and Views
- Spring's MVC is very flexible
- Encourages 2 or more application services in a single controller, i.e., one method for one service.
- *In Servlets - one service per request*
- Unlike Struts in Spring MVC there are no action forms. It binds directly to the domain objects.
- In Spring MVC, **DispatcherServlet** class works as the front controller which controls other controller classes.



Spring MVC – Handler Mapping

HandlerMapping is an interface

Its implementing classes will map the request to the corresponding method of a controller

```
@RequestMapping("/welcome.html")
public ModelAndView helloWorld(HttpServletRequest request, HttpServletResponse response) {

    String username = request.getParameter("username");
    return new ModelAndView("hellopage", "username", username);
}
```

`return new ModelAndView("hellopage", "username", username);`



Before View Resolving ---> hellopage.jsp

After View Resolving ---> hellopage.html

```
<servlet-name>spring</servlet-name>
<url-pattern>*.html</url-pattern>
```

@Controller : A simple Java class to handle requests is declared as a Controller.

```
@Controller  
public class ProductController {
```

@RequestMapping : The [@RequestMapping](#) annotation in Spring MVC is used to map HTTP requests to specific handler methods in a Spring MVC Controller class.

```
@Controller  
public class ProductController {  
    @RequestMapping("/product")  
    public ModelAndView login(HttpServletRequest request, HttpServletResponse response)
```

@RequestParam:

The **@RequestParam** annotation is used in Spring MVC for handling HTTP request parameters. (Instead Servlet's `request.getParameter()` method)

@RequestParam annotation enables Spring MVC to extract input data that may be passed as form data.

***** Efficient without using `request.getParameter()`. Automatic mapping with the form parameters.**

Eg.

```
@RequestMapping("/productrqpm")
```

```
public ModelAndView login(@RequestParam("productName") String productName,  
                          @RequestParam("productBrand") String productBrand) {
```

```
<form>
```

```
<input type="text" name="productName" />
```

```
<input type="text" name="productBrand" />
```

**** Demo**

=====

@PathVariable :

The **@PathVariable** annotation in Spring MVC is used to extract values from the URL path and bind them to method parameters in a controller.

Eg. The path variable below is {name}.

Path variable and parameter name should match

```
@RequestMapping("/str/{name}")
public ModelAndView mapPathVariableByName(@PathVariable String name) {
    return new ModelAndView("welcome", "username", name);
}
```

**** Demo**

@ModelAttribute :

The **@ModelAttribute** annotation in Spring MVC is used to bind request parameters to a Java object and automatically add that object to the model.

It simplifies form handling by eliminating the need to manually create objects and add them to the Model.

Eg.

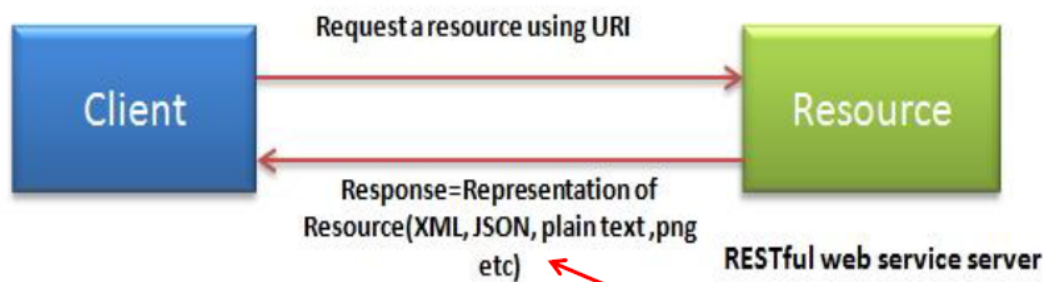
```
@ModelAttribute
public Country getCountry(@RequestParam String countryName,
                          @RequestParam long population) {
    // avoiding request.getParameter() to receive form data
    Country country = new Country();
    country.setCountryName(countryName);
    country.setPopulation(population);
    return country;
}
```

**** Demo**

REpresentational State Transfer

RESTful Web Services - JAX-RS

Clients and servers exchange representations of resources



The response from server is considered as the representation of the resources.

=====

RESTful Services Benefits:

Language and Platform independent: RESTful web services can be written in any programming language and executed in any platform.

Permits different data format: RESTful web service permits different data format such as Plain Text, HTML, XML and JSON.

=====

JSON stands for JavaScript Object Notation

{JSON is a lightweight format for storing and transporting data

```
{  
    "id" : 1001,  
    "name" : "Kumar"  
    .....  
}
```

JSON is often used when data is sent from a web application to a web page

Data on modern web browsers are internally in JSON format. So only data rendering is lightening fast.

=====

RESTful resources

REST allows that resources have different representations

A resource is accessed via a common interface based on the HTTP standard methods – GET, POST, PUT, DELETE like CRUD operations

POST creates a new resource

GET defines a reading access of the resource

PUT updates an existing resource

DELETE removes a resource